BusBidder Spain: Personalized Bus Journey Comparison and Booking System

Kyle Strother

November 2, 2026

Contents

1 Introduction **2**

2 Requirements and Problem Interpretation **2**

3 System Architecture and Software Design **3**

3.1 Data Flow Diagram 3

3.1.1 Trip Data Ingestion and Storage 4

3.1.2 Priority Evaluation and Ranking System 4

4 Data Management and Validation **4**

5 Search, Filtering and Weighted Ranking **4**

6 Web Interface and Booking Functionality **5**

7 Testing and Experimental Results **5**

8 Traveller Profile Comparisons **5**

9 Critical Evaluation and Limitations **6**

1 Introduction

- Introduce BusBidder Spain and its purpose.
- Explain the problem of comparing competing bus offers.
- Establish the objective: personalized filtering and weighted ranking of journeys.
- Briefly summarize the application's functionality.

2 Requirements and Problem Interpretation

- Identify the main functional requirements.
- Explain key design assumptions and decisions.
- Distinguish mandatory requirements from user preferences.
- Summarize how the implementation addresses the assignment.

3 System Architecture and Software Design

3.1 Data Flow Diagram

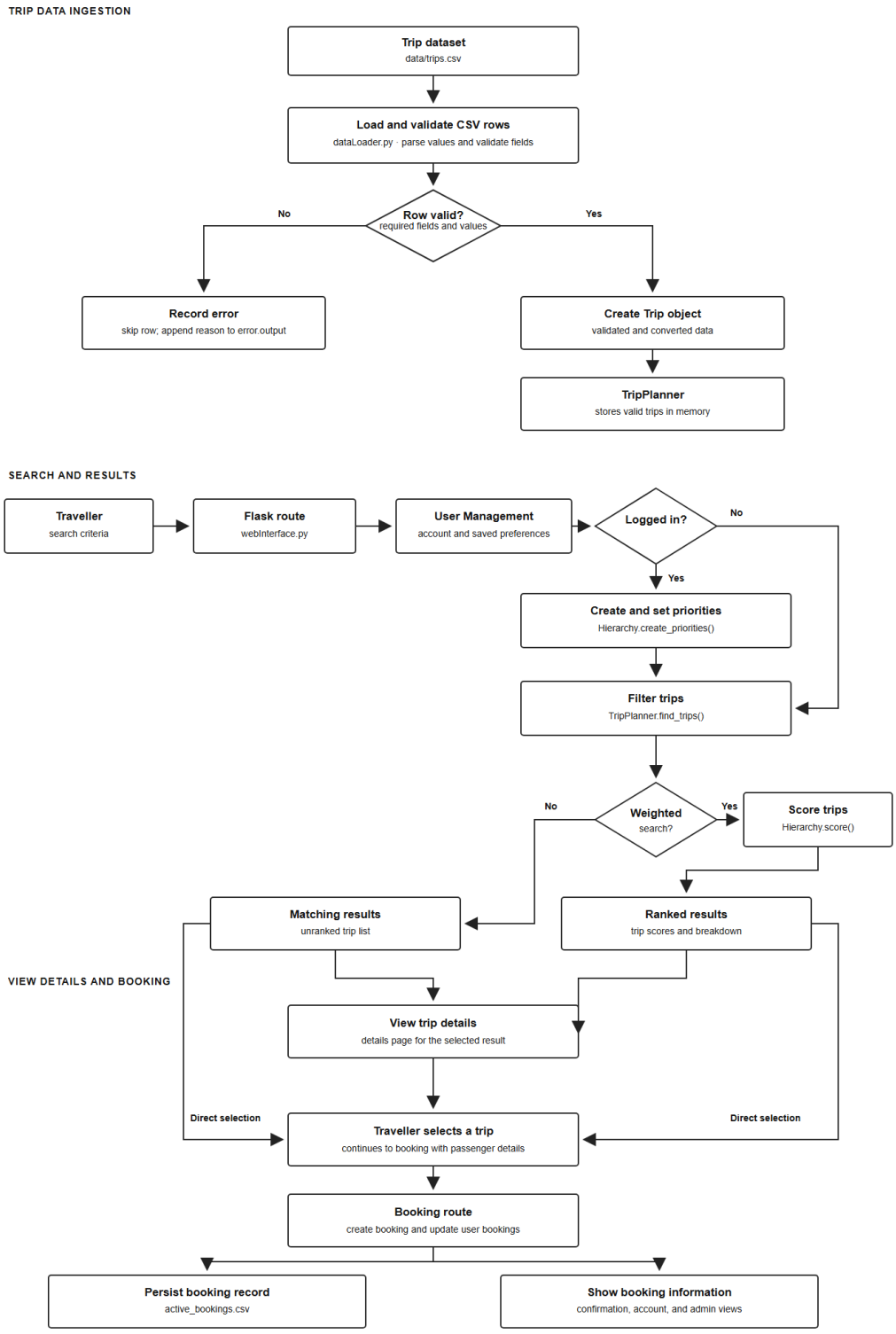


Figure 1: BusBidder data flow from trip ingestion through ranking and booking.

3.1.1 Trip Data Ingestion and Storage

The project structure includes two key modules for reading data and storing it for application processing. The file `dataLoader.py` is responsible for loading the trip data via an CSV data file `data/trips.csv`. The `dataLoader.py` validates it row by row rejecting and reporting failed imports along with reasoning for rejection before they are converted into `Trip` objects. The file `tripsService.py` then manages the storage of `Trip` objects, and then implements the system for querying stored trips, and is also responsible for exporting booking information to external files.

3.1.2 Priority Evaluation and Ranking System

The module `priority.py` defines priority types, qualifiers, and evaluation methods. The module `priorityHierarchy.py` stores the user's chosen weights and ranks candidate trips. The file `scoreObject.py` contains the objects used to represent a trip's score and the breakdown of each priority contribution. The class definitions in `user.py` handle the user account model and saved preferences, while `webInterface.py` exposes the application routes and page rendering logic.

This architecture reflects object-oriented design principles. Each class is responsible for a limited and well-defined set of responsibilities, which makes the code easier to test. The use of separate modules also allows the ranking system to operate independently from the web interface, which is useful for both validation and future extension. Inheritance, composition, counting logic, and dynamic behaviour are all relevant to the project. For example, a user can maintain a priority hierarchy that changes at runtime, and trip scoring can adapt to different priority types without changing the wider application model.

The application also demonstrates composition through the relationship between a user and their saved priorities, and between a trip and its underlying attributes. The design allows multiple traveller profiles to be evaluated with the same underlying trip dataset, which is a major strength of the system.

- Explain the modular project structure and module responsibilities.

- Introduce the main classes and their relationships.

- Justify object-oriented design decisions.

- Discuss relevant inheritance, composition, decorators and dynamic behaviour.

4 Data Management and Validation

- Describe the dataset, its structure and contents.

- Explain CSV loading and type conversion.

- Describe validation of invalid, missing and inconsistent records.

- Explain error handling, custom exceptions and logging.

- Discuss how invalid records are handled.

5 Search, Filtering and Weighted Ranking

- Explain the search and filtering process.

- Describe hard constraints versus soft preferences.
- Explain the Priority and PriorityHierarchy architecture.
- Describe comparative versus binary evaluation methods.
- Explain score normalization and the weighting system.
- Discuss flexibility scoring and qualifiers.
- Present the overall scoring formula.
- Explain score transparency and ranking results.

6 Web Interface and Booking Functionality

- Describe the Flask application and its structure.
- Explain the main user-facing functionality.
- Discuss registration, authentication and saved preferences.
- Describe the booking workflow and booking persistence.
- Briefly explain frontend implementation (HTML, CSS, JavaScript).

7 Testing and Experimental Results

- Describe the testing approach and automated test suites.
- Present data validation tests.
- Present scoring and ranking tests.
- Discuss boundary, invalid and missing-data cases.
- Include screenshots and selected application outputs.
- Evaluate whether observed results are reasonable.

8 Traveller Profile Comparisons

- Define three different traveller profiles:

- **Budget-focused traveller**: prioritises low price, low transfer count, and value for money.
- **Business traveller**: prioritises time efficiency, reliability, and preferable departure times.
- **Flexibility-focused tourist**: prioritises refundable tickets, luggage allowance, and adaptable travel arrangements.

- Present their respective priority weights.
- Run all three against the same dataset.
- Compare their resulting rankings and scores.
- Explain why their recommendations differ.

9 Critical Evaluation and Limitations

- Identify the application's main strengths.
- Discuss technical limitations and design trade-offs.
- Evaluate scoring assumptions and normalization.
- Discuss potential unfair outcomes and unintended biases.
- Identify possible improvements and future development.
- Reflect on challenges and lessons learned.